Tab 1

DANN Cost

MySQL Adaptation & DANN Results Report

LightGBM Base Model · 5-Fold CV · 4-Domain & 2-Domain Classification

1. Overview

1.1 DANN Dataset — Domain Breakdown

The DANN experiments use a merged dataset of **10,320 samples** (125 features after processing) split into either 4 fine-grained domains or 2 coarse engine-level domains.

	MYSQL-32	MYSQL-64	POSTGRES-32	POSTGRES-64	
Setup	**Domain 0**	**Domain 1**	**Domain 2**	**Domain 3**	**Total**
4-domain	1,409	719	1,212	6,980	**10,320**
2-domain	2,128		8,192		**10,320**

Global train/val split: 8,772 samples. Test set: 1,548 samples. Test domain counts: d0=211, d1=108, d2=182, d3=1,047.

1.2 Normalization Strategies

Global Normalization

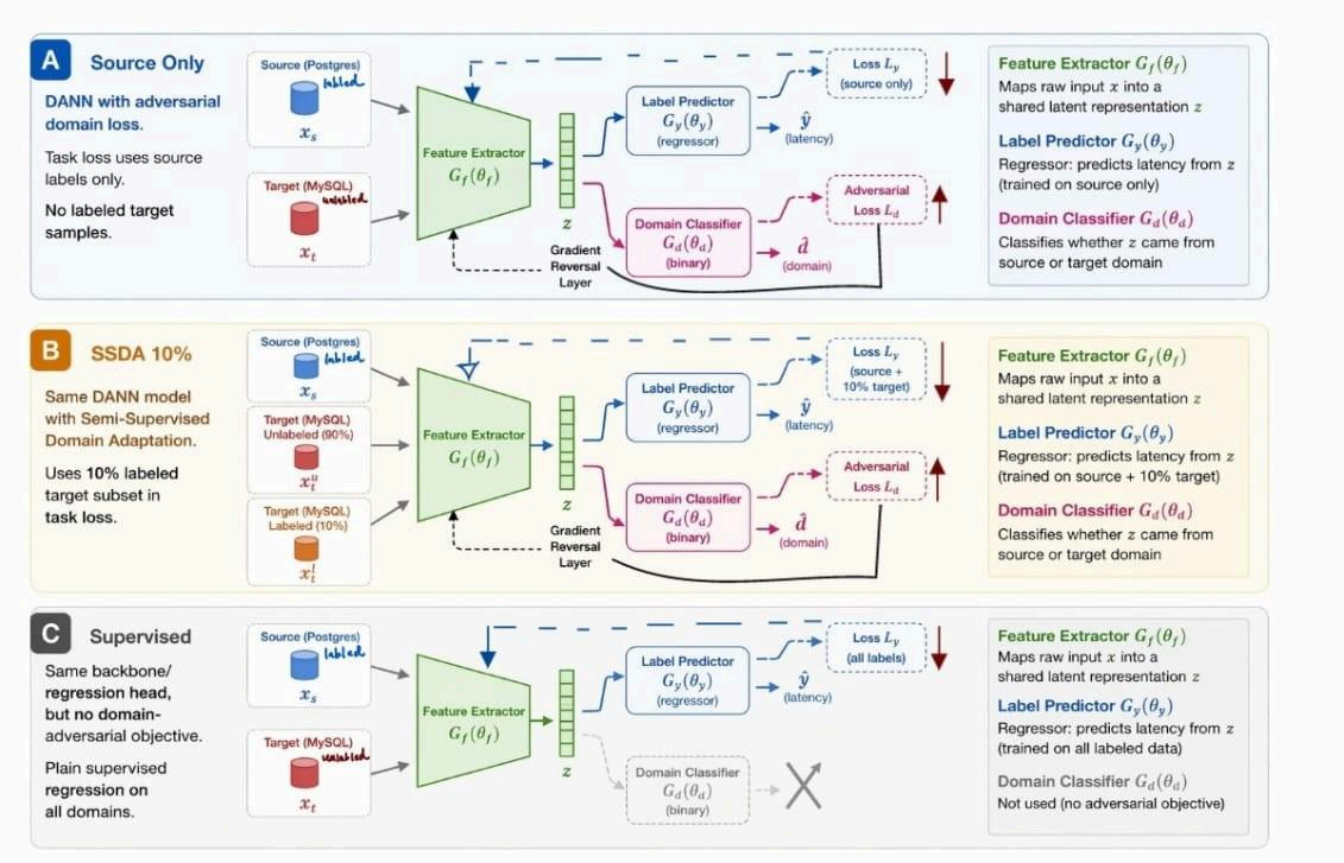
A single mean and standard deviation is computed over all rows. Every sample is normalized using the same global pair, and denormalization also uses the same global pair.

Per-Engine Normalization

Separate mean and standard deviation are computed for each database engine. Each row is normalized using its own engine's statistics, and denormalization uses that row's engine stats.

1.3. Experiment Setup (A / B / C)

Label	Name	Description	alpha_target
A	Source Only (Unsupervised)	DANN with adversarial domain loss. Task loss uses source labels only. No labeled target samples.	0.0
B	SSDA 10%	Same DANN model with Semi-Supervised Domain Adaptation. Uses 10% labeled target subset in task loss.	0.3
C	Supervised	Same backbone/regression head, but no domain-adversarial objective. Plain supervised regression on all domains.	—



All three experiments use the same backbone and regression head. Seeds 42, 52, and 62 are used for robustness evaluation.

2. DANN Experiment Results — 4-Domain Classification

Robust summary across seeds (mean ± std). Dataset shape: (10,320, 125). Domain IDs: 0 (MySQL-32, 1,409), 1 (MySQL-64, 719), 2 (PostgreSQL-32, 1,212), 3 (PostgreSQL-64, 6,980).

2.1 Global Normalization

Rank	Experiment	RMSE ± std	NRMSE %	MAPE %	Spearman ρ	Dom. Acc
1	**C — No DANN Supervised**	4.18 ± 0.90	6.51 ± 1.14%	165.31 ± 70.51%	0.7988 ± 0.0390	0.1032
2	B — DANN SSDA 10%	7.53 ± 0.63	11.79 ± 1.08%	339.69 ± 22.16%	0.5281 ± 0.0203	0.4022
3	A — DANN Source Only	9.05 ± 1.30	14.15 ± 1.86%	81.97 ± 19.33%	0.5696 ± 0.2675	0.3732

⚠ *C wins all seeds under global normalization. B is ~80% worse on RMSE. A has unusually high domain accuracy (0.37) but the worst RMSE, confirming domain accuracy, does not predict cost regression quality.*

2.2 Per-Engine Normalization

Rank	Experiment	RMSE ± std	NRMSE %	MAPE %	Spearman ρ	Dom. Acc
1	**C — No DANN Supervised**	5.75 ± 0.85	9.00 ± 1.31%	84.78 ± 26.26%	0.8472 ± 0.0493	0.3752
2	B — DANN SSDA 10%	6.74 ± 0.67	10.52 ± 0.59%	84.88 ± 20.83%	0.8162 ± 0.0818	0.7921
3	A — DANN Source Only	13.02 ± 4.75	20.32 ± 7.18%	227.85 ± 74.77%	0.7000 ± 0.0335	0.6339

⚠ *C remains rank 1. A collapses to RMSE 13.02 ± 4.75 and MAPE 228% — the highest variance of any configuration — confirming source-only DANN is incompatible with per-engine normalization.*

3. DANN Experiment Results — 2-Domain Classification

The same dataset (10,320 samples, 125 features) collapsed to 2 engine-level domains: MySQL (domain 0, 2,128 samples) and PostgreSQL (domain 1, 8,192 samples). Train/val: 8,772; test: 1,548 (d0=319, d1=1,229).

3.1 Global Normalization

Rank	Experiment	RMSE ± std	NRMSE %	MAPE %	Spearman ρ	Dom. Acc
1	**C — No DANN Supervised**	4.81 ± 1.35	6.77 ± 3.42%	196.27 ± 113.00%	0.5896 ± 0.2965	0.1017
2	A — DANN Source Only	8.88 ± 0.91	11.83 ± 2.75%	43.71 ± 3.57%	0.7043 ± 0.1226	0.5305
3	B — DANN SSDA 10%	8.94 ± 3.35	12.58 ± 7.29%	796.79 ± 860.82%	0.3309 ± 0.1242	0.6357

⚠ *B degrades severely under 2-domain global normalization — MAPE 796.79% ± 860.82%, the highest instability observed across all configurations. C remains rank 1. A improves in rank vs. the 4-domain setting, suggesting coarser domain labeling benefits the source-only approach slightly.*

3.2 Per-Engine Normalization

Rank	Experiment	RMSE ± std	NRMSE %	MAPE %	Spearman ρ	Dom. Acc
1	**C — No DANN Supervised**	5.65 ± 0.79	7.48 ± 1.61%	81.45 ± 22.45%	0.8504 ± 0.0068	0.2008
2	B — DANN SSDA 10%	6.19 ± 0.94	8.68 ± 3.63%	103.65 ± 31.80%	0.8328 ± 0.0256	0.9711
3	A — DANN Source Only	11.14 ± 2.54	14.48 ± 2.39%	142.42 ± 69.01%	0.7453 ± 0.0699	0.7605

⚠ *B achieves near-perfect domain accuracy (0.971) under 2-domain per-engine normalization but still cannot match C on RMSE or Spearman. C leads with Spearman ρ 0.850 and MAPE 81.45%.*

4. MySQL Adaptation Results (5-Fold CV)

Three adaptation strategies are compared, each built on top of the best-performing PostgreSQL base model (LightGBM). All results are mean ± std across 5 folds.

4.1 Model Configurations

- MySQL-only Regressor — trained exclusively on MySQL samples; no transfer from source domain.
- Residual Adapter — learns a residual correction on top of the frozen PostgreSQL base model predictions.
- Joint Weighted Training — retrains jointly on PostgreSQL + MySQL data with class/domain weighting.

4.2 5-Fold Cross-Validation Results

Model Configuration	RMSE	MAPE (%)	Spearman ρ
MySQL-only Regressor	3.3853 ± 0.6655	12.26 ± 5.85	0.4693 ± 0.0437
Residual Adapter	3.3402 ± 0.6729	5.93 ± 0.73	0.5681 ± 0.0431
Joint Weighted Training	**3.1431 ± 0.5958**	**4.81 ± 0.23**	**0.6118 ± 0.0359**

The bold green row indicates the best-performing configuration. Joint Weighted Training achieves the lowest RMSE (3.14), lowest MAPE (4.81%), and highest Spearman ρ (0.612) across all three strategies.

> ⚠ *MAPE improvement from MySQL-only (12.26%) to Joint Weighted Training (4.81%) represents a 61% relative reduction, indicating strong benefit from source-domain transfer.*

2.3 Key Observations

- Joint Weighted Training dominates on all three metrics — RMSE, MAPE, and Spearman rank correlation.
- The Residual Adapter achieves a small RMSE improvement over MySQL-only but yields a large MAPE reduction (12.26% → 5.93%), suggesting it corrects systematic per-query biases.
- All three configurations have similar RMSE variance (±0.60–0.67), indicating stable performance across folds.
- Spearman ρ improves markedly from 0.469 (MySQL-only) to 0.612 (Joint Weighted), showing better cost rank ordering — critical for query plan selection.

5. Cross-Experiment Comparison & Conclusion

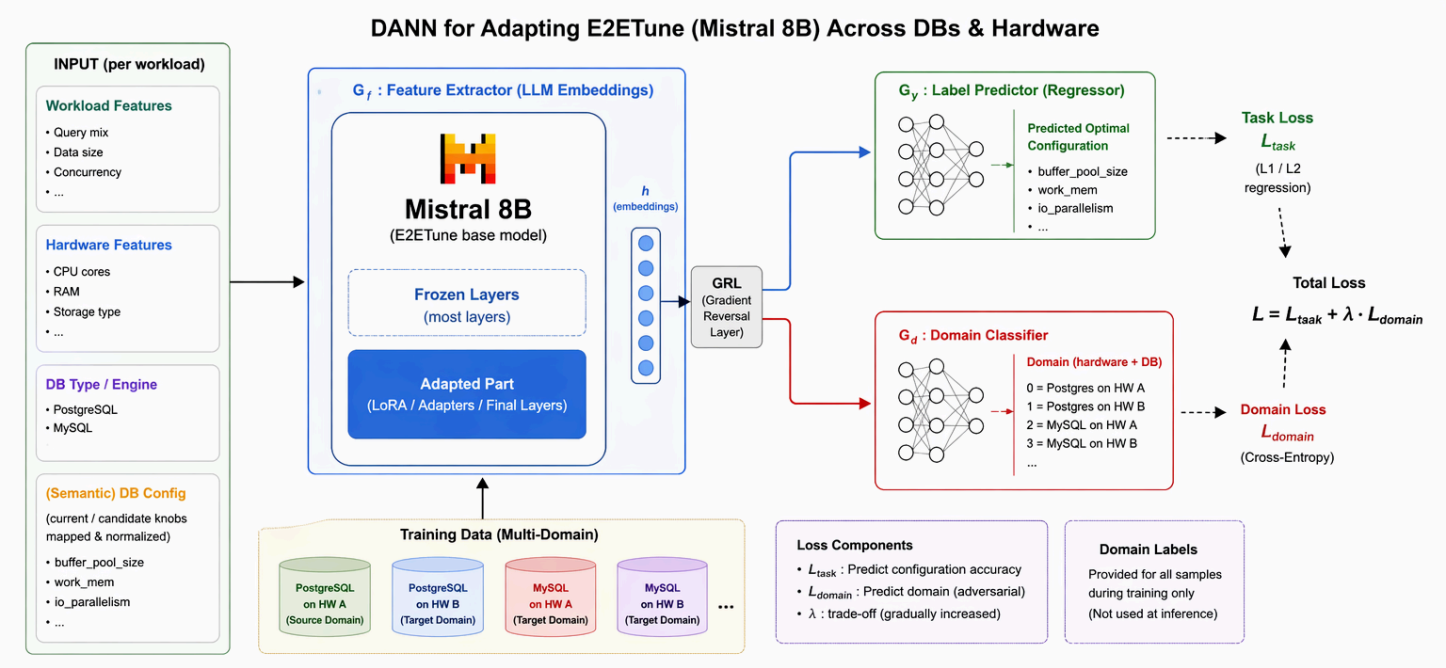
5.1 Best Configuration per Setting

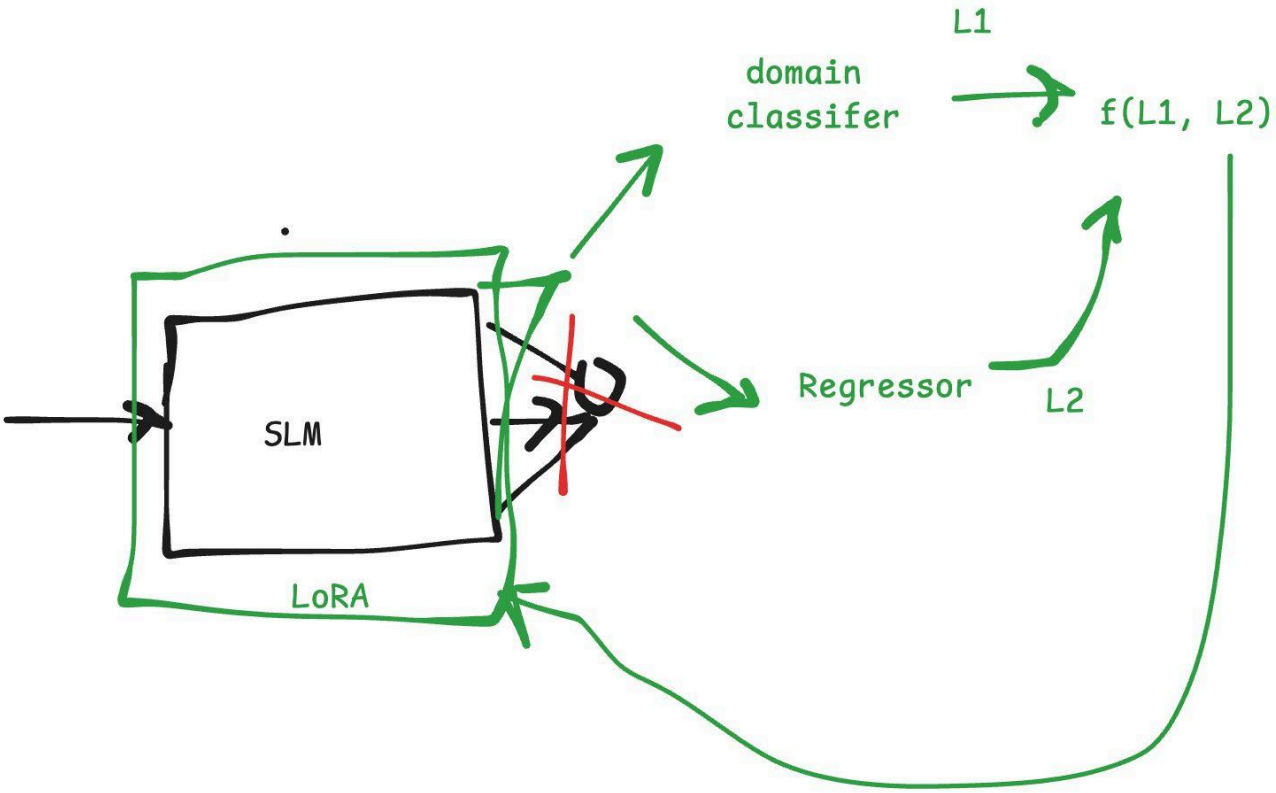
Setting	Best Model	RMSE	MAPE %	Spearman ρ
MySQL Adaptation (5-fold CV)- Light GBM	**Joint Weighted Training**	3.1431 ± 0.5958	4.81 ± 0.23	0.612 ± 0.036
DANN 4-domain, global	**C — No DANN Supervised**	4.18 ± 0.90	165.31	0.799 ± 0.039
DANN 4-domain, per_engine	**C — No DANN Supervised**	5.75 ± 0.85	84.78	0.847 ± 0.049
DANN 2-domain, global	**C — No DANN Supervised**	4.81 ± 1.35	196.27	0.590 ± 0.297
DANN 2-domain, per_engine	**C — No DANN Supervised**	5.65 ± 0.79	81.45	0.850 ± 0.007

5.2 Key Findings

- Joint Weighted Training is the strongest MySQL adaptation strategy — it leverages PostgreSQL source data while down-weighting it relative to the scarce MySQL samples, achieving the best RMSE, MAPE, and Spearman across all 5 folds.
- Plain supervised regression (C) consistently dominates DANN-based approaches across all normalization modes and domain granularities. Domain adversarial training does not improve cost prediction in this setting.
- The source-only DANN (A) is the weakest approach in most configurations — especially under per-engine normalization — where it exhibits the highest RMSE and largest variance across seeds.
- B (SSDA 10%) sometimes closes the gap with C (notably in 4-domain per-engine and 2-domain per-engine settings) but never surpasses it on RMSE.
- 2-domain classification does not consistently improve over 4-domain. B degrades catastrophically on MAPE under 2-domain global normalization (797% vs. 340%), while C and A remain stable.
- Domain classification accuracy is not a reliable proxy for regression quality. B and A regularly achieve higher domain accuracy than C while producing worse RMSE and Spearman.
- Per-engine normalization consistently produces higher Spearman ρ for C and B vs. global normalization, suggesting it helps rank ordering even when it does not reduce absolute RMSE.

Next Step:




L1
domain
classifer
f(L1, L2)
Regressor
L2
SLM
LoRA

Tab 2

Domain Adaptation

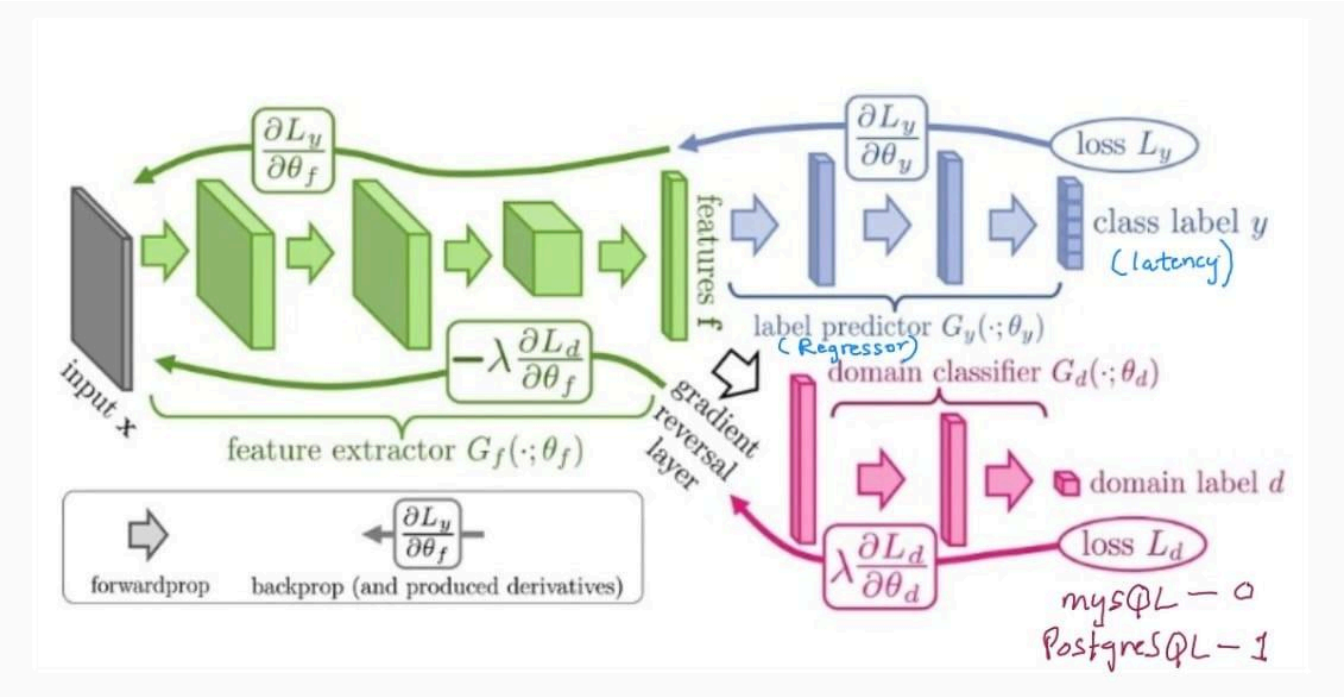
for Cost Model Training using DANN

1. Domain-Adversarial Neural Networks (DANN)

DANN is a transfer learning framework that enables a model trained on a labeled source domain to generalize to an unlabeled (or sparsely labeled) target domain. It does so by simultaneously optimizing task performance while making learned features domain-invariant.

1.1 Architecture Components

DANN consists of three modules that are trained jointly:



Module	Symbol	Role
Feature Extractor	*Gf (θf)*	Maps raw input x into a shared latent representation z Input Features: • **Query Plan Embeddings** • **Knob Configurations** • **Workload Features** • **Hardware Specs:** RAM, CPU cores, and threads
Label Predictor	*Gy (θy)*	Regressor: predicts latency from z
Domain Classifier	*Gd (θd)*	Classifies whether z came from source or target domain • **DB Type Flag:** 0 (PostgreSQL) or 1 (MySQL)

1.2 Training Objective

DANN optimizes a minimax objective that balances **task accuracy(minimize error)** and **domain confusion(maximize error):**

$$E(\theta f, \theta y, \theta d) = (1/n) \Sigma Ly(\theta f, \theta y) - \lambda \cdot [(1/n) \Sigma Ld(\theta f, \theta d) + (1/n') \Sigma Ld(\theta f, \theta d)]$$

Where:

- Ly — Task-specific loss (e.g., MSE for regression)
- Ld — Domain classification loss (binary cross-entropy: source vs. target)
- λ — Hyperparameter controlling the trade-off between task performance and domain invariance

The Gradient Reversal Layer (GRL) sits between Gf and Gd. During backpropagation it negates the gradient by −λ, forcing Gf to produce features that confuse the domain classifier while still minimizing task loss.